

Computação Natural: Trabalho Prático 2

Otimização por Colônia de Formigas

Pedro Ramos
{*pedroramos@dcc.ufmg.br*}

15 de Novembro de 2015

1 Introdução

O algoritmo da otimização da colônia de formigas (ACO, do inglês ant colony optimization algorithm), introduzido por Marco Dorigo em sua tese de PhD é uma heurística baseada em probabilidade, criada para solução de problemas computacionais que envolvem procura de caminhos em grafos. Este algoritmo foi inspirado na observação do comportamento das formigas ao saírem de sua colônia para encontrar comida.

Na natureza, as formigas andam inicialmente sem rumo até que, encontrada comida, elas retornam à colônia deixando um rastro de feromônio. Se outras formigas encontram um desses rastros, elas tendem a não seguir mais caminhos aleatórios. Em vez disso, seguem a trilha encontrada, retornando e inclusive liberando mais feromônio se acharem mais alimento.

Com o transcorrer do tempo, entretanto, as trilhas de feromônio começam a evaporar, reduzindo, assim, sua força atrativa. Quanto mais formigas passarem por um caminho determinado, mais tempo será necessário para que o feromônio associado àquela trilha evapore. Analogamente, elas marcharão mais rapidamente por sobre um caminho curto, o que implica **aumento da densidade de feromônio depositado antes que ele comece a evaporar**. A evaporação do feromônio também possui a vantagem de evitar a convergência para uma solução local ótima: se a evaporação não procedesse, todas as trilhas escolhidas pelas primeiras formigas se tornariam excessivamente atrativas para as outras e, neste caso, a exploração do espaço da solução se delimitaria consideravelmente.

Todavia, quando uma formiga encontra um bom caminho entre a colônia e a fonte de alimento, outras formigas tenderão a seguir este caminho, gerando assim **feedback positivo**, o que eventualmente torna um determinado caminho mais interessante. A idéia do algoritmo da colônia de formigas é imitar este comportamento através de "formigas virtuais" que caminham por um grafo que

por sua vez representa o problema a ser resolvido.

Apesar de ter sido inicialmente abstraído para produzir soluções quase ótimas para o problema do caixeiro viajante, neste trabalho aplicou-se o ACO como uma metaheurística para o problema do Maior Caminho.

2 Implementação

2.1 Representação do problema

Como dito anteriormente, aplicou-se o ACO para resolver o problema do Maior Caminho em um grafo. O problema do maior caminho é NP Completo, enquanto a metaheurística que usa o ACO opera em tempo polinomial.

O trabalho foi desenvolvido na linguagem Python v. 2.7. O grafo foi representado como um **ndarray** do **numpy**. Dessa forma o algoritmo ACO implementado recebe, como entrada, um grafo ponderado direcionado que será armazenado em uma matriz de adjacências, assim como o nó de partida e o nó de término do caminho cujo comprimento deseja-se maximizar. Como saída, o ACO retorna um caminho válido (que parta do nó inicial e vá até o final) juntamente com o seu comprimento.

2.2 Representação do ACO

O ACO opera de maneira análoga ao aplicável ao caixeiro viajante. A única diferença reside na função de avaliação de uma solução. No ACO, a solução avaliada é o caminho percorrido pela formiga. Para o problema do Maior Caminho, aquela formiga que percorrer o maior caminho será aquela a liberar mais feromônio, para atrair as outras formigas para convergência daquela solução.

Dessa forma, o ACO segue o seguinte pseudo-algoritmo para o problema do maior caminho:

```
for cada iteração do  
  inicializa todas as formigas em nós aleatórios;  
  for formiga em COLONIA do  
    formiga constrói um caminho;  
  end for  
  avaliação dos caminhos das formigas da colônia;  
  N melhores formigas depositam feromônio no caminho percorrido;  
end for
```

A construção do caminho da formiga é um procedimento local e guloso. A escolha do próximo nó a ser incorporado na solução é baseada em uma função de distribuição de probabilidade que leva em consideração não só o aspecto local

mas também o global. A probabilidade da formiga ir para um nó X depende não só o do PESO associado ao caminho para chegar-se a este nó mas também do FEROMÔNIO contido neste caminho. Dessa forma leva-se em consideração o aspecto local (escolha gulosa, qual o nó mais distante) e o global (qual aquele caminho que tem mais feromônio). Considerando-se ambos, chegou-se a esta função de probabilidade abaixo. A probabilidade de ir do nó i ao nó j é:

$$P_{ij} = \frac{Pw_{ij} + P\rho_{ij}}{2} \quad (1)$$

Em que Pw é a probabilidade de acordo com o peso, e $P\rho$ é a probabilidade de acordo com o feromônio.

$$Pw_{ij} = \frac{W_{ij}}{\sum_{j=0}^N W_{ij}} \quad (2)$$

$$P\rho_{ij} = \frac{\rho_{ij}}{\sum_{j=0}^N \rho_{ij}} \quad (3)$$

A fitness da solução é basicamente avaliada como o comprimento do caminho percorrido pela formiga. A cada iteração, aquela que percorreu o maior caminho em relação as outras formigas, é aquela que vai depositar mais feromônio. Assim como o grafo foi representado por uma matriz de adjacências, o feromônio também é representado como uma matriz análoga contendo as quantidades de feromônio entre os vértices. A quantidade máxima de feromônio que uma formiga pode soltar é 100, e a mínima é 10. A quantidade máxima de feromônio que pode estar em um caminho no grafo é 100, e a mínima é 0.1.

A cada iteração, antes de atualizar os feromônios, a matriz de feromônios é evaporada de acordo com a **taxa de evaporação**. A taxa de evaporação é um parâmetro ajustável ao ACO, e que controla a convergência. Uma taxa muito baixa faz o algoritmo convergir rapidamente, pois as formigas vão escolher caminhos com alto teor de feromônio. Uma taxa muito alta deixa o algoritmo aleatório e não há progressão na solução.

A inicialização da colônia é feita da seguinte maneira: o número mínimo de formigas da colônia é 10. Dessa forma, 10 formigas iniciam no nó inicial; as outras N-10 são espalhadas aleatoriamente pelos nós do grafo e iniciam em pontos aleatórios. **Algumas formigas não provém uma solução válida.** Isso ocorre pois o grafo é direcionado e uma das premissas para se achar o maior caminho é que não haja nós repetidos no caminho (caso contrário, o maior caminho seria infinito), de forma que uma formiga pode acabar 'presa' em um nó sem poder retornar para um caminho que a leve ao nó final, e sem poder continuar a construção da solução sem repetir nenhum nó já antes visitado. Uma outra razão é quando a formiga inicia em um nó que não é o inicial, e termina a solução no nó final, porém seu caminho não representa uma solução válida.

Quando isso ocorre, a solução inválida é considerada no momento de avaliação para decidir quais formigas liberarão feromônio. Caso haja uma solução inválida que supere todas as outras soluções, inválidas ou não, naquela iteração, a formiga responsável por aquele caminho depositará o feromônio normalmente, e o algoritmo continuará a executar.

Para evitar que, ao final do ACO, não haja nenhuma solução viável, duas medidas foram tomadas. A primeira, já dita, de colocar 10 formigas no nó inicial, forçando assim que aquelas formigas comecem do primeiro nó. Mesmo que elas não cheguem ao nó final, as chances de que gerem uma solução válida são maiores do que se elas iniciassem em um nó aleatório. A segunda medida foi construir uma solução em busca em profundidade caso não haja solução válida ao final do programa. Felizmente, mais de 100 testes foram executados e em nenhum momento o programa deixou de gerar uma solução válida. Isso ocorreu devido ao fato de que muitas iterações e muitas formigas foram geradas a cada execução, aumentando a probabilidade de serem geradas soluções válidas.

3 Experimentos

3.1 Base de Dados

Para os experimentos foram utilizadas 3 bases de dados diferentes, disponibilizadas em arquivos de texto simples contendo o número de nós e os pesos das arestas direcionadas do grafo. Para duas entradas sabe-se a solução ótima, enquanto que para uma das entradas não se sabe a solução ótima.

Entrada	Nº Nós	Solução Ótima
Entrada 1	100	990
Entrada 2	20	168
Entrada 3	1000	?

3.2 Variação de Parâmetros

Após exaustivos testes sobre as entradas 1 e 2, cujas soluções são conhecidas, chegou-se a 4 configurações (settings) de parâmetros (variando-se 1 a cada configuração). A variação dos parâmetros entre essas configurações provocou diferentes efeitos. Ao se aumentar a taxa de evaporação, percebeu-se uma queda na qualidade da solução. Acredita-se ser devido à aleatoriedade provocada por uma alta taxa de evaporação, que anula o aspecto global influenciador na escolha do próximo nó a percorrer.

O a diminuição no número de formigas de 100 para 10 não provocou muitas alterações na solução. Por outro lado, a alteração no número de iterações foi drástica, indicando que o mais importante no ACO não é o número de formigas, e sim o número de iterações, para que as formigas consigam achar o melhor

caminho a partir do feedback das formigas anteriores.

Abaixo vemos uma tabela com detalhes de cada configuração, bem como figuras que demonstram a qualidade das soluções das entradas 1 e 2 em cada configuração. A configuração indicada para este algoritmo implementado é a primeira (Setting 1). Cada configuração foi executada **30 vezes** para cada entrada. Os testes foram rodados em ambiente Mac OS X 10.10.5 Yosemite, CPU Intel Core i5 2.9GHz e 3MB de Cache, 8GB de RAM DDR3, e SSD de 512GB.

Configuração	Nº Formigas	Nº Iterações	Taxa de Evaporação
Config 1	100	1000	0.1
Config 2	100	100	0.1
Config 3	10	1000	0.1
Config 4	100	1000	0.6

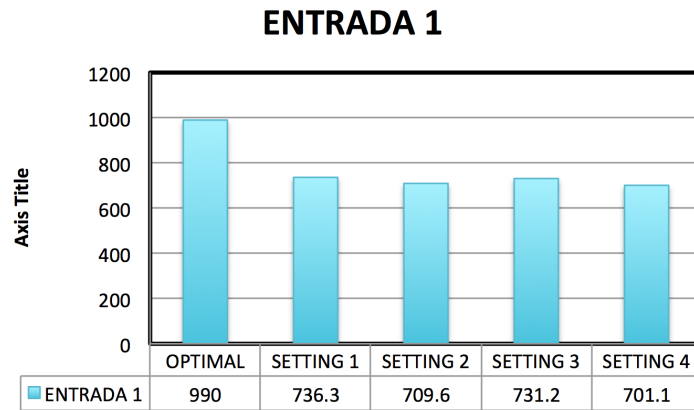


Figura 1: Execuções sobre a Entrada 1

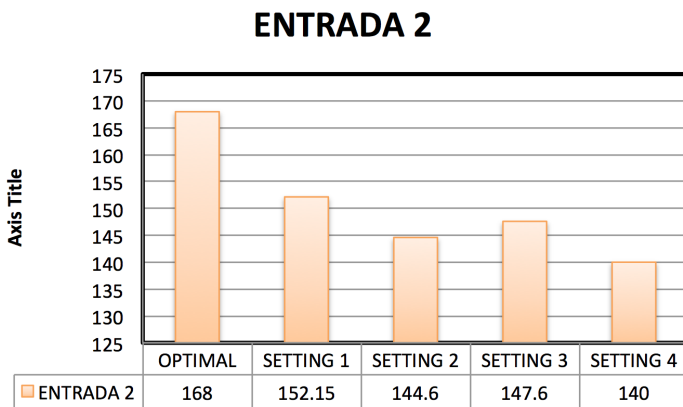


Figura 2: Execuções sobre a Entrada 2

Por fim, a Figura 3 mostra resultados da execução das 4 configurações sobre a entrada 3. A melhor solução obtida foi 5539.

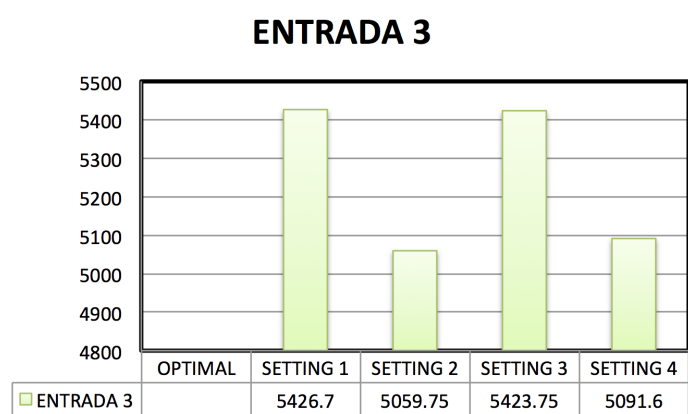


Figura 3: Execuções sobre a Entrada 3

Referências

- [1] M. Dorigo, V. Maniezzo e A. Colorni, *The Ant System: Optimization by a colony of cooperating agents*, 1996.
- [2] M. Dorigo, *Optimization, Learning and Natural Algorithms*, Politecnico di Milano, Italy. 1992.
- [3] M. Dorigo, *Ant Colony Optimization*, Scholarpedia, 2007.
- [4] Wan-Shiou Yang e Shi-Xin Weng, *Application of the Ant Colony Optimization Algorithm to the Influence-Maximization Problem*, 2012.